

C h a i n o f R e s p o n s i b i l i t y

Chain of Responsibility Pattern

Pass the request down a chain

DEFINITION

Pass a request along a chain of handlers; each decides to handle it or forward it.

Chain of Responsibility gives several objects a shot at a request, lined up so each handler either deals with it or forwards it to the next. The sender drops the request at the head and never learns who answers; membership and order are assembled at runtime. It turns a sprawling conditional into a pipeline you can reorder and extend.

WHY IT MATTERS

Hard-wiring "who handles what" couples the sender to every possible receiver and buries the logic in one growing if/else. A chain decouples them: each handler knows only its successor, so you add, remove, or reorder handling without touching the sender. The cost is traceability — a request can fall off the end unhandled, and the runtime path is not obvious from the source.

— How it works

Handler	Defines handle() and usually holds the next link in the chain.
ConcreteHandler	Handles the requests it can; otherwise forwards to its successor.
Client	Builds the chain and sends the request to the first handler.

HOW TO APPLY

1. List the handlers and the order they should see the request.
2. Give each a handle(req) that either responds or delegates to the next.
3. Link them head to tail; the client holds only the head.
4. Add a default tail handler so nothing falls off the end unanswered.

LITMUS TEST

Could more than one object handle this, with the right one unknown until runtime? If you are growing an if/else over receiver types, make each branch a handler.

— Smell → fix

One method hard-codes every check in order, binding the sender to all of them:

```
// one method owns every rule, in order
if (!req.user) return '401'
if (req.banned) return '403'
return serve(req) // a new rule edits this code
```

↓ LINK HANDLERS INTO A CHAIN

```
abstract class Handler {
  next?: Handler
  setNext(h: Handler) { this.next = h; return h }
  handle(req) { return this.next?.handle(req) }
}
class Auth extends Handler {
  handle(req) { return req.user ? super.handle(req) : '401' }
}
```

Each handler now knows only its successor, so you reorder or insert rules without touching the others. Add a terminal handler when a response is required, or a request can fall off the end.

USE WHEN

- ✓ Middleware pipelines
- ✓ Escalation / approval flows; event filtering

AVOID WHEN

- ▲ Exactly one known handler — just call it

— Trade-offs & pitfalls

PROS

- + Decouples sender from receiver
- + Add or reorder handlers freely

CONS

- No guarantee the request is handled
- Long chains are hard to debug

COMMON PITFALLS

- ▲ A request no handler accepts silently falls off the end — add a default tail when a response is required.
- ▲ Order is behavior: reordering handlers changes outcomes, so make the sequence explicit, not incidental.
- ▲ Hard to debug — the runtime path is not visible in the source; log the hops. Don't build a chain just to look extensible.

WHERE YOU SEE IT

- ▶ Express / Koa / NestJS middleware, guards and interceptors — each layer handles or calls next().
- ▶ Servlet / ASP.NET filter pipelines that a request passes through.
- ▶ DOM event bubbling: an event walks up its ancestors until handled or stopped.

SEE ALSO

Command	handlers frequently carry Command objects down the chain
Decorator	same linked-wrapper shape — but a Decorator always delegates, a Chain may stop early
Mediator	the alternative when peers coordinate through a hub instead of a line
Composite	a parent node often acts as the successor when chaining over a tree

— Interview & sources

Chain vs Decorator?

Same linked shape; a Decorator always delegates and adds behavior, a Chain handler may stop the request.

Is Express middleware a chain?

Yes — each handler processes, then calls `next()`: the textbook Chain of Responsibility.

What if no one handles it?

It falls off the end; add a default / terminal handler when a response is required.

Short-circuit vs pass-through?

Short-circuit stops at the first capable handler; pass-through runs every handler as a pipeline.

REMEMBER

A request walks a line of handlers until one handles it — the sender never names the receiver.

SOURCES

GoF — "Design Patterns" (1994): Chain of Responsibility (behavioral)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Chain of Responsibility